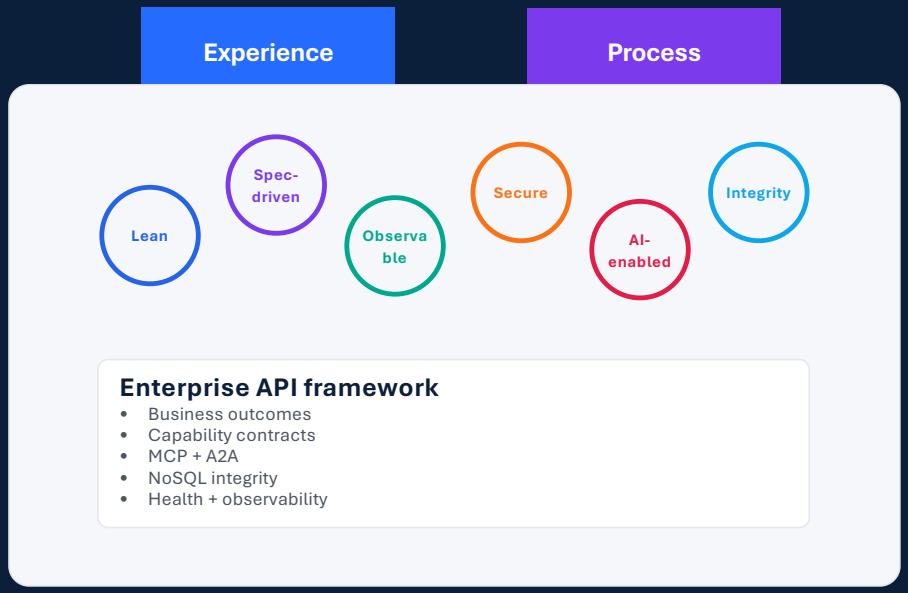
Outcome-Centric API Design

A framework for lean, spec-driven, AI-enabled, observable, and integrity-preserving enterprise APIs

Conference presentation

API layers as capability contracts

Framework design → implementation approach → cap & grow adoption → hypothetical results



Bhagyeshkumar Chokhawala | Comcast Cable

MOTIVATION

Why API architecture needs a stronger operating model

Enterprise API portfolios often grow faster than their governance, testing, observability, and transition practices.

Endpoint sprawl

Screen-specific and database-wrapper APIs multiply across teams.

Fragile processes

Partial failures leave mismatched states across business domains.

AI risk surface

Agents and tools can invoke APIs without sufficient guardrails.

NoSQL integrity gap

Denormalized entities and projections need explicit consistency controls.

Adoption friction

Teams need a practical path to cap legacy entropy and grow target capabilities.

Architecture question: How can APIs stay lean, secure, observable, AI-ready, and transaction-safe while spanning multiple domains and data stores?

CONTRIBUTION

The paper’s contribution: a measurable API architecture framework

It connects design intent to implementation evidence, operational telemetry, and transition governance.

1. Design intent
Outcome, capability, resource, contract

2. Spec evidence
OpenAPI/AsyncAPI, policies, examples

3. Implementation
API layers, orchestration, data integrity

4. Runtime proof
Tests, observability, health, compliance

5. Adoption path
Cap legacy sprawl, grow target API products

Core contribution

API design is treated as a governance system: each API must provide value, contract clarity, authority control, test evidence, observability, and integrity status.

Talk flow: framework design → implementation approach → cap & grow transition → hypothetical evaluation

FRAMEWORK DESIGN

Layered API architecture: capability contracts, not endpoint inventory

Each layer has a distinct responsibility, ownership model, observability contract, and integrity obligation.

Experience API

Channel, persona, journey
GET /agent-desktop/...

Process API

Workflow across domains
POST /customer-disconnect-requests

Domain API

Reusable capability
GET /orders/{orderId}

System API

Legacy/system of record
GET /billing-system/...

Data API

Governed data products
GET /customer-profile/{id}

Event API

Publish/consume events
OrderSubmitted

AI Tool API

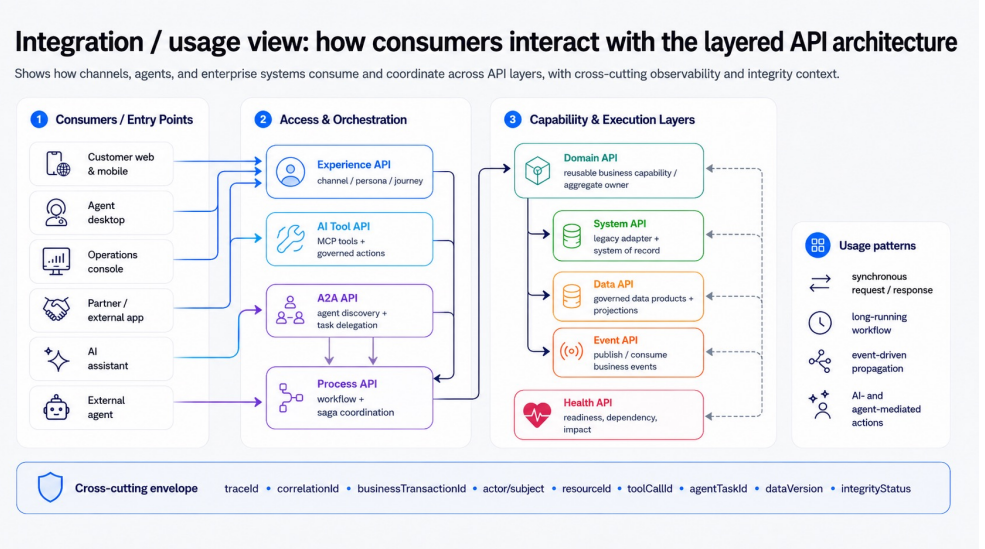
Approved MCP tools
getOrderDetail

Health API

Readiness + impact
GET /health/capabilities

A2A API

Agent discovery + tasks
delegateBillingReview



Cross-cutting envelope: traceId • correlationId • businessTransactionId • actor/subject • resourceId • toolCallId • agentTaskId • dataVersion

FRAMEWORK DESIGN

Capability contract dimensions: what every API must prove

The framework defines a minimum evidence set for design review, implementation, runtime, and compliance.

Outcome

Business value and measurable result

Capability

Owning domain and reusable behavior

Contract

Spec, schema, error model, examples

Resource

Collection/detail access patterns

Lean

Right-sized payloads and surface area

Security

Actor, subject, resource, action, context

AI/MCP/A2A

Tool access and agent collaboration

Testing

Contract, policy, mock, failure paths

Observability

Trace, metrics, logs, audit evidence

Health

Readiness and business impact

NoSQL Integrity

Aggregate, version, saga, reconciliation

Compliance

Framework adherence and release gate

Design rule: every API must demonstrate business value, right-sized design, explicit authority, traceable execution, test evidence, health impact, and integrity status.

IMPLEMENTATION APPROACH

Spec-driven API factory: from intent to release gate

A repeatable delivery loop reduces ambiguity, accelerates development, and turns governance into automation.

1. Intent
outcome + capability + owner

2. Spec
OpenAPI/AsyncAPI + policies

3. Mock
scenario data + virtualization

4. Build
API + orchestration + data guardrails

5. Test
contract + auth + failure paths

6. Observe
traces + health + compliance evidence

Release gates

- Spec linting and breaking-change detection
- Policy-as-code for role and object-level authorization
- Mock coverage for happy path, failure path, and edge cases
- Telemetry contract present before production release

IMPLEMENTATION APPROACH

Reference implementation pattern: runtime components and responsibilities

Implementation converts framework dimensions into concrete runtime components.

Gateway

authn, rate limit, token validation

Policy Engine

actor/subject/resource/action decisions

Experience API

journey-oriented aggregation

Process API

saga, state, idempotency

Domain APIs

aggregate ownership, business rules

Data/Event APIs

projections, outbox/inbox, freshness

MCP/A2A Gateway

tool and agent governance

Observability Plane

traces, metrics, logs, audit, health

Implementation principle

Keep business authority and integrity in deterministic APIs. AI and A2A agents may assist, but execution remains tool-scoped, policy-controlled, idempotent, and fully audited.

IMPLEMENTATION APPROACH

Lean resource design: generic patterns before special endpoints

A lean API exposes only useful business surface area while preserving navigability, pagination, and evolvability.

Scoped collection
GET /{parent}/{id}/{resources}

Filtered collection
GET /{resources}?filter=...

Resource detail
GET /{resources}/{resourceId}

Example: order access as a generic pattern

```
GET /customers/{customerId}/orders
GET /accounts/{accountId}/orders
GET /orders/{orderId}
```

Lean guardrails

- Summary list ≠ full detail payload
- One internal query capability; multiple clear contracts
- Avoid duplicate endpoint variants for the same purpose
- Measure payload weight, reuse, defects, and consumer friction

IMPLEMENTATION APPROACH

Role-aware authorization: same API, different authority modes

The same business API may be executed under different actor-subject relationships and policy controls.

User self-service
actor = subject
role + ownership + eligibility

Assisted / impersonated
actor ≠ subject
assist permission + subject entitlement

Operational override
elevated role
reason + approval + audit

System initiated
service identity
workflow state + event correlation

Policy decision inputs

- actor
- subject
- resource
- action
- relationship
- context
- business state
- risk
- reason code
- approval

Audit trail must preserve true actor, effective subject, acting mode, policy decision, and impacted resource.

IMPLEMENTATION APPROACH

Transactional and NoSQL data integrity across API layers

NoSQL flexibility shifts integrity from hidden database assumptions to explicit API-layer ownership and reconciliation.

Process API

durable state
saga step status
idempotency key

Domain API

aggregate owner
single writer
version/ETag

System API

legacy consistency
retry contracts
fallback behavior

Data API

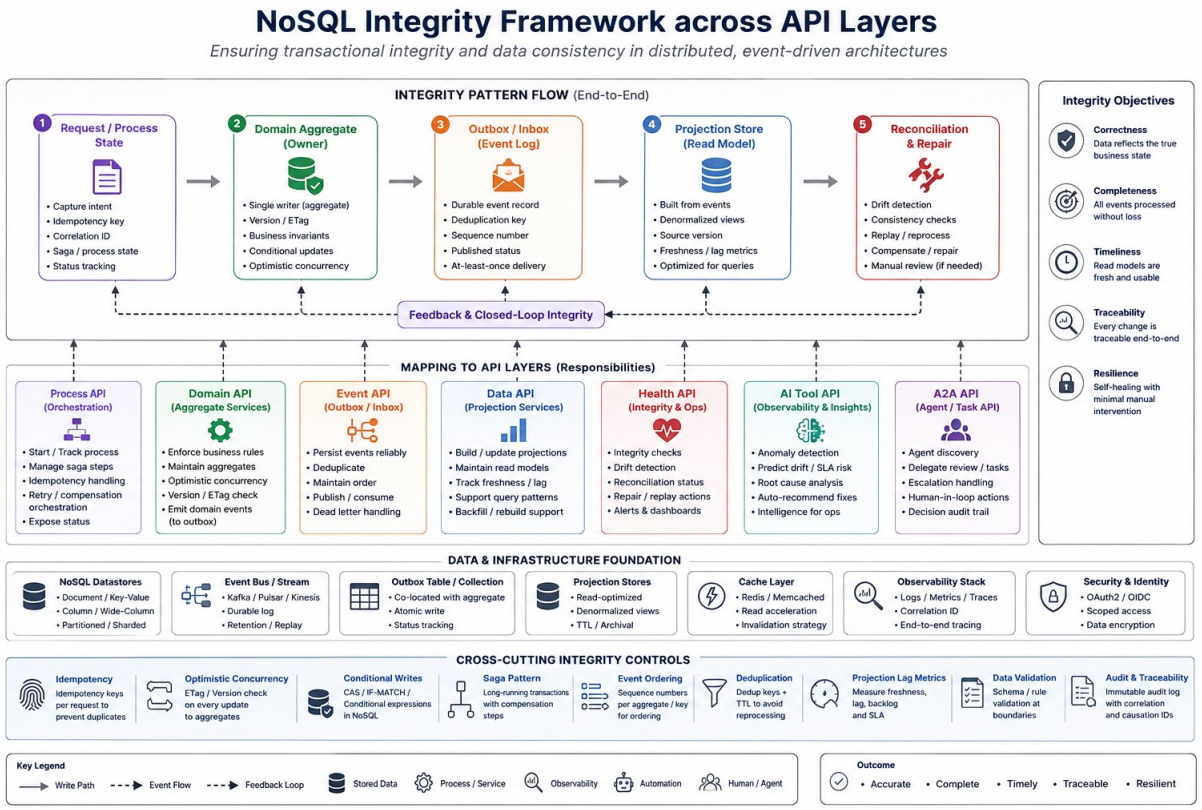
projection freshness
source version
lineage

Event API

outbox/inbox
deduplication
sequence/ordering

Health API

integrity status
lag + fallout
impact advice



Example pattern: `GET /{transactionRequests}/{requestId}/integrity` exposes entity versions, projection lag, event delivery status, reconciliation state, and manual-action flags.

IMPLEMENTATION APPROACH

AI-enabled APIs: MCP for tools, A2A for agent collaboration

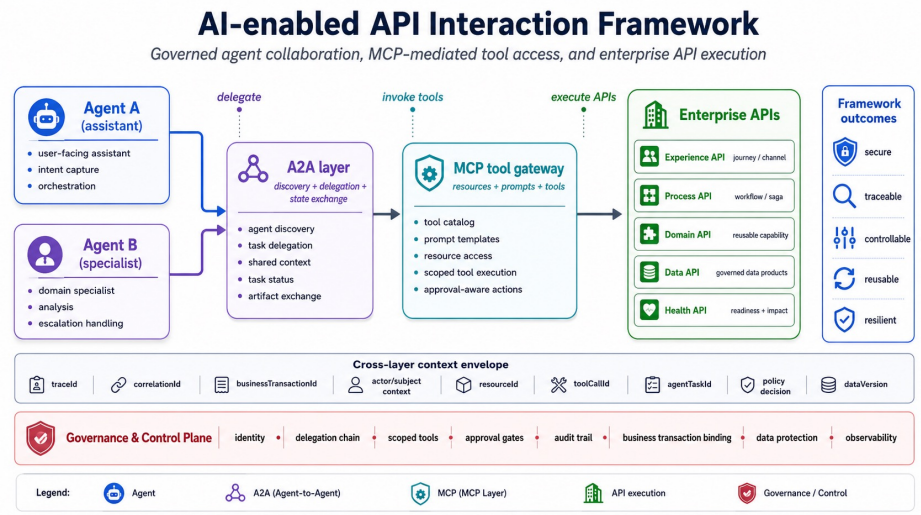
AI should interact through governed interfaces, not by bypassing API policy and transaction controls.

AI Assistant
user-facing reasoning

MCP Tool Gateway
agent → tool/API

Approved APIs
getOrderDetail • checkReadiness

A2A Collaboration
agent → agent tasks



Responsibility boundary

- MCP: controlled access to APIs, tools, prompts, and resources
- A2A: governed agent discovery, task delegation, state exchange, and artifacts
- Process API: deterministic execution, idempotency, authorization, and audit

High-risk guardrails

- Scoped tools and agent identity
- Human approval for disconnect/refund/settlement
- Tool-call and agent-task audit
- No direct database or unrestricted execution

IMPLEMENTATION APPROACH

Observability and health: business trace across every API layer

Observability is the evidence system for policy, integrity, SLOs, health, and adoption.

Experience | Process | Domain | System | Data

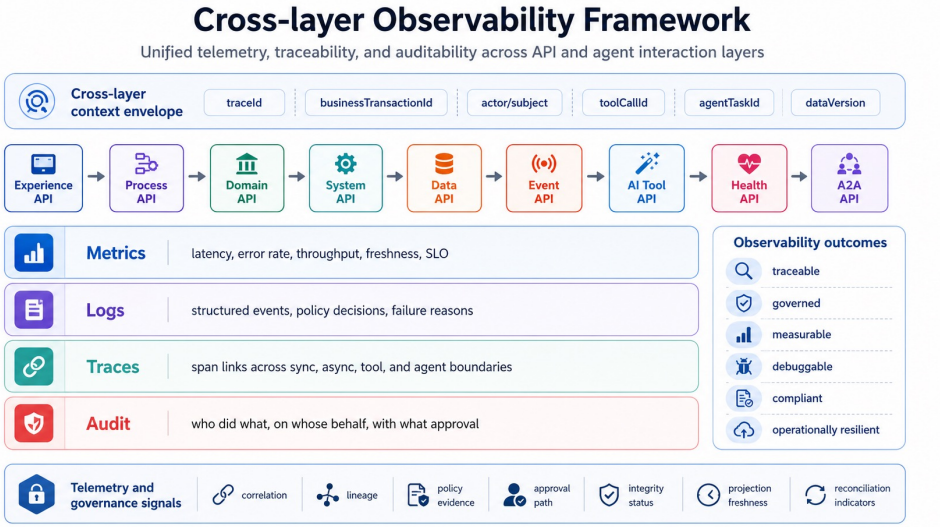
Event | AI Tool | Health | A2A

Trace envelope

traceId	correlationId	businessTransactionId
actor/subject	resourceId	toolCallId
agentTaskId	projectionVersion	integrityStatus

Health as a consumer contract

```
/health/live
/health/ready
/health/dependencies
/health/capabilities
/health/impact
```



Outcome signal: “API is up” becomes “business transaction is progressing, authorized, consistent, observable, and recoverable.”

EVALUATION

Experimental design: 10 APIs, 4 workflows, 5 consumer types

Experimental results illustrate how the framework could be evaluated in a controlled enterprise pilot.

Scope

10 APIs across order, account, billing, device, data, event, AI tool, A2A, and health layers

Consumers

Agent desktop, workflow engine, AI assistant, operations console, integration service

Workflows

Order lookup, disconnect, billing review delegation, projection reconciliation

Measures

Reuse, contract defects, auth findings, fallout, MTTR, projection freshness, compliance score

Experimental setup

Baseline implementation compared to framework-applied implementation using the same functional scope and simulated transaction volume. Results are illustrative, not empirical.

EXPERIMENTAL RESULTS

Experimental outcomes: quality improved without slowing delivery

Experimental pilot result: the largest gains came from spec-driven contracts, policy-as-code, failure-path tests, and cross-layer telemetry.

+41 pts

API reuse score

baseline 42 → 83

-71%

contract defects

31 → 9 defects

-67%

transaction fallout

12% → 4%

-63%

MTTR

5.4h → 2.0h

0

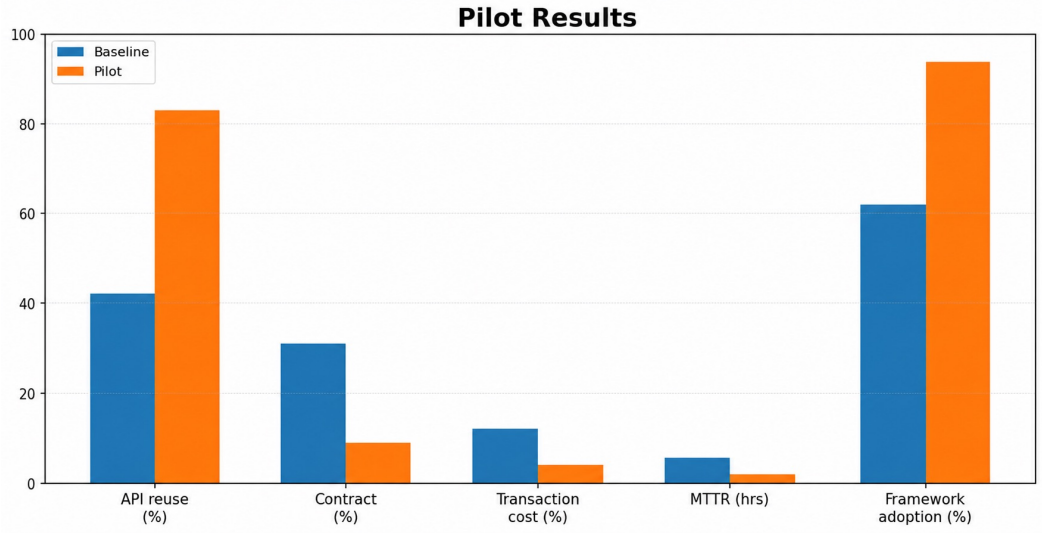
high-risk AI bypasses

policy gate blocked all

94%

framework adherence

release gate score



Interpretation

Framework adherence created measurable evidence: every API had a spec, test evidence, policy decision model, trace envelope, health impact, and integrity status.

ADOPTION APPROACH

Cap & Grow: transition from API sprawl to governed capability contracts

Cap what creates risk today; grow the target architecture through measurable adoption waves.

CAP

1. Stop the entropy

Freeze new screen-specific and database-wrapper endpoints unless exempted by architecture review.

2. Classify and score

Tag existing APIs by layer, owner, consumer, risk, reuse, contract quality, observability, and integrity evidence.

3. Wrap or retire

Use facades/BFFs for unstable legacy access, then retire duplicate or low-value endpoints.

GROW

4. Build API products

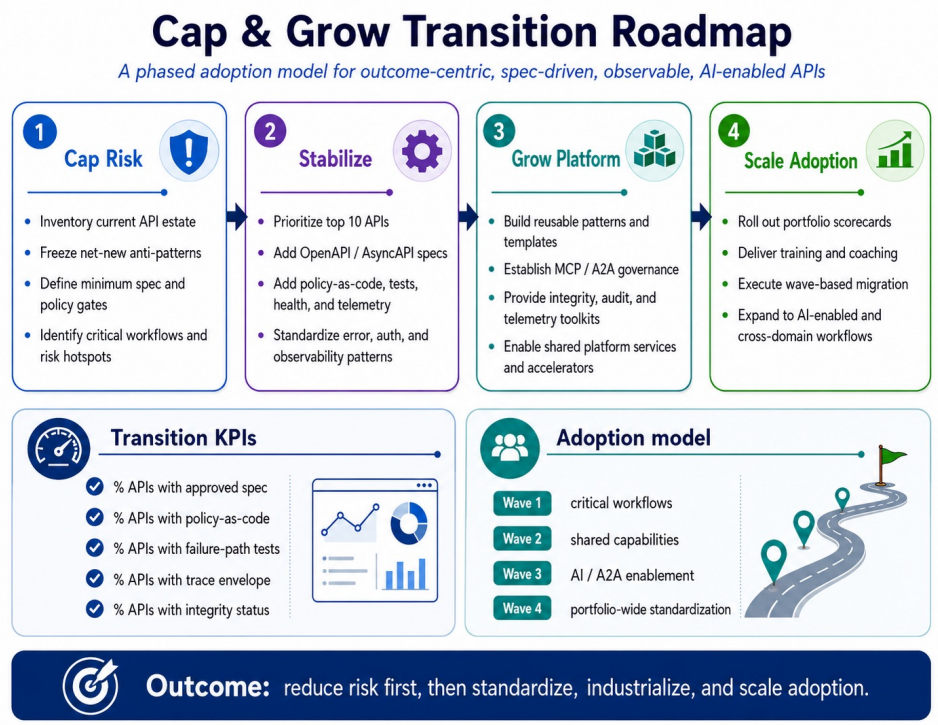
Grow domain, process, data, event, health, MCP, and A2A APIs using the spec-driven factory.

5. Automate compliance

Make framework adherence a CI/CD release gate: specs, tests, policy, health, trace, and integrity.

6. Scale by capability

Expand wave-by-wave: order lookup → disconnect workflow → billing review → AI/A2A-assisted operations.



Closing thesis: APIs should be managed as business capability contracts — lean, spec-driven, secure, observable, AI-ready, and integrity-preserving.

Q&A

Thank You

Research Profile QR Code

Scan the QR code to open my research profile, publications, Google Scholar, ORCID, and professional links.